



# Simulación de Eventos Discretos

Miguel Ángel Rincón Morales<sup>1</sup>

1. Cod: 160005038, Ing. Sistemas.

Facultad de Ciencias Básicas e Ingenierías. Programa de Ingeniería de Sistemas.

Fecha: 30 de Septiembre de 2026

---

## Resumen

Este informe documenta el desarrollo de la práctica de simulación de eventos discretos (SED). Se implementaron en Python, siguiendo la metodología de Rios08, tres modelos de sucesos discretos: una línea de espera con un servidor ( $G/G/1$ ), la red de colas de una estación de inspección técnica de vehículos y un modelo de inventario probabilístico con política  $(p, P)$ . Toda la aleatoriedad proviene del generador congruencial mixto de módulo  $2^{48}$  validado en los laboratorios anteriores, con los generadores de variables exponenciales (transformada inversa) y normales (método polar) ya construidos. Para el  $G/G/1$  se simuló dos escenarios con  $T = 40$  y se repitió cada uno 10 veces; para la red de colas, dos escenarios con  $T = 300$  y 20 repeticiones con intervalos de confianza del 95 %. Ambos escenarios del  $G/G/1$  y el escenario A de la red resultaron saturados ( $\rho > 1$ ), lo que se refleja en tiempos medios en el sistema que crecen con el horizonte y en  $T_p$  comparables con  $T$ ; el escenario B de la red se autorregula gracias al servicio acelerado del nodo 3 cuando  $n_3 \geq 5$ . El inventario con  $p = 20$  obtuvo un beneficio medio de 17 643 con un 84,6 % de clientes satisfechos y un 12,6 % del tiempo sin existencias; el análisis de sensibilidad mostró que subir el punto de pedido a  $p \approx 40$  maximiza el beneficio. Por último, se recorrió el tutorial de SimPy y se implementaron sus siete ejemplos oficiales, y se construyeron y ejecutaron en AnyLogic PLE los cinco modelos de los tutoriales de la guía, identificando con sus estadísticas integradas el cuello de botella de cada sistema.

**Palabras clave** — simulación de eventos discretos, lista de sucesos, línea de espera  $G/G/1$ , red de colas, inventario probabilístico, intervalos de confianza, SimPy, AnyLogic.

---

## 1. Objetivos

- Introducir al paradigma de simulación de eventos discretos por medio de sus elementos, diseño, metodología de modelado e implementación.
- Apropiar la metodología de modelado e implementación de simulación de eventos discretos a partir de dos casos de uso: un sistema de línea de espera con un servidor y una red de colas.
- Implementar la metodología de modelado y simulación de eventos discretos en un caso de estudio de un modelo de inventario probabilístico.
- Implementar casos de estudio de simulación de eventos discretos en frameworks o softwares de simulación para eventos discretos como SimPy y AnyLogic.



## 2. Fundamento teórico

Los conceptos que siguen corresponden a la presentación de clase 6. *Simulación de eventos discretos* [1] y al Capítulo 5 de [2], secciones 5.1 a 5.6.

### 2.1. El paradigma de simulación de sucesos discretos

Un *sistema* es un conjunto de entidades que interactúan para cumplir un fin; su *estado* es el conjunto de variables necesario para describirlo en un instante dado. En un sistema de sucesos discretos el estado cambia únicamente en instantes aislados del tiempo, cuando ocurre un *suceso* (una llegada, el fin de un servicio, la llegada de un pedido). Entre dos sucesos consecutivos el estado permanece constante, de modo que el reloj de la simulación no avanza de forma continua sino que salta de suceso en suceso. Los elementos del modelo son [2]:

- **Variables de estado:** describen el sistema (número de clientes en cada nodo, nivel de inventario).
- **Reloj de simulación  $t$ :** instante actual.
- **Lista de sucesos  $TSuc$ :** instante programado para la próxima ocurrencia de cada tipo de suceso; un valor centinela  $M$  muy grande indica que ese suceso no está programado.
- **Variables contador** ( $N_{LL}$ ,  $N_S$ , ...): cuántas veces ha ocurrido cada suceso.
- **Variables acumuladoras:** integran una magnitud a lo largo del tiempo (por ejemplo, la suma de  $n \times (\text{duración})$  para obtener el número medio de clientes).
- **Rutinas de suceso:** una por tipo de suceso; actualizan el estado, los contadores y acumuladores, y programan nuevos sucesos.
- **Variables de salida:** medidas de desempeño calculadas al terminar ( $t_{med}$ ,  $T_p$ ).

La metodología de implementación es siempre la misma: (i) inicializar  $t = 0$ , el estado, los contadores y  $TSuc$ ; (ii) generar el primer suceso; (iii) mientras haya sucesos programados, extraer el de menor instante, avanzar el reloj hasta él y ejecutar su rutina; (iv) calcular las variables de salida. Como una ejecución produce una única realización de variables aleatorias, es necesario *replicar* la simulación con flujos independientes y analizar las réplicas con técnicas estadísticas.

### 2.2. Sistema de cola simple $G/G/1$

Un único servidor atiende clientes que llegan con tiempos entre llegadas i.i.d. de distribución  $F$  y requieren tiempos de servicio i.i.d. de distribución  $G$ , con disciplina FIFO y cola infinita. Las entradas se permiten hasta el horizonte  $T$ , pero se sigue atendiendo hasta vaciar el sistema. Los sucesos son dos: *Llegada* y *Servidor* (fin de servicio). Las salidas de interés son el tiempo medio en el sistema, el tiempo medio en cola, el tiempo  $T_p$  desde  $T$  hasta que sale el último cliente, el número máximo simultáneo de clientes y el total atendido. La relación  $\rho = \lambda/\mu$  entre la tasa de llegadas y la de servicio determina el comportamiento: si  $\rho < 1$  el sistema es estable y las medidas convergen; si  $\rho \geq 1$  la cola crece sin límite y las medidas dependen de  $T$  [2].



### 2.3. Red de colas

En la Inspección Técnica de Vehículos (ITV) de la Sec. 5.5.2 de [2] los vehículos llegan según un proceso de Poisson de parámetro  $\lambda$  y pasan por una ventanilla (nodo 1, servicio  $\mathcal{N}(\mu_1, \sigma_1)$  truncada); con probabilidad 0,4 requieren pruebas adicionales en una nave auxiliar (nodo 2, servicio  $\mathcal{Exp}(\mu_2)$ ) y después todos pasan por las pruebas estándar (nodo 3), cuyo tiempo de servicio depende de la congestión:  $\mathcal{N}(\mu_{31}, \sigma_{31})$  si  $n_3 < 5$  y  $\mathcal{N}(\mu_{32}, \sigma_{32})$  con  $\mu_{32} < \mu_{31}$  en caso contrario. Hay cuatro sucesos (*Llegada\_cliente*, *Servicio\_nodo1*, *Servicio\_nodo2*, *Servicio\_nodo3*) y la lista  $TSuc = \{t_{LL1}, t_{S1}, t_{S2}, t_{S3}\}$ . El número medio de clientes en cada nodo se obtiene acumulando  $n_j \times (t_{suc} - t)$  en cada suceso y dividiendo por el tiempo total; el tiempo medio en el sistema, como los clientes pueden adelantarse unos a otros, se calcula como la suma de los tiempos medios por nodo ponderando el nodo 2 por 0,4.

### 2.4. Modelo de inventario probabilístico

Un almacén vende un producto a  $r$  por unidad a clientes que llegan según un proceso de Poisson de parámetro  $\lambda$  y demandan una cantidad aleatoria. La política de revisión continua  $(p, P)$  (Figura 1 de la guía) lanza un pedido de  $P - x$  unidades, a un costo  $h$  por unidad, cuando el nivel  $x$  cae por debajo del punto de pedido  $p$  y no hay pedidos pendientes; el pedido llega  $L$  unidades de tiempo después. Mantener inventario tiene un costo por unidad y tiempo, y la demanda que no puede atenderse se pierde. Los sucesos son la llegada de un cliente y la llegada de un pedido; las salidas son el beneficio (ingresos menos costos de pedidos y de almacenamiento), el porcentaje de clientes satisfechos y el porcentaje de tiempo con inventario en cero.

### 2.5. SimPy

SimPy [3] es una librería de simulación de eventos discretos para Python orientada a *procesos*: cada entidad activa es un generador que cede eventos (`yield env.timeout(d)`, `yield recurso.request()`) al entorno **Environment**, que mantiene la lista de sucesos y reanuda cada proceso cuando su evento ocurre. Los recursos compartidos **Resource** (servidores con cola FIFO), **PreemptiveResource** (con prioridades e interrupciones), **Container** (cantidades continuas) y **Store** (objetos) modelan la mayoría de los sistemas de colas sin programar explícitamente la lista de sucesos.

### 2.6. AnyLogic

AnyLogic [4] es un entorno gráfico de simulación multimétodo (eventos discretos, agentes y dinámica de sistemas). Para eventos discretos ofrece la *Process Modeling Library*, cuyos bloques (**Source**, **Queue**, **Delay**, **Service**, **SelectOutput**, **Sink**) se conectan en un diagrama de flujo; sobre ella se construyen librerías especializadas para peatones (*Pedestrian Library*), tráfico vial (*Road Traffic Library*) y manejo de materiales (*Material Handling Library*). Los modelos se animan en 2D/3D y recogen estadísticas con histogramas y gráficos de series.

## 3. Procedimiento o metodología

Las secciones 5.1 a 5.4 se implementaron en Python en cuatro Notebooks de Jupyter ejecutados en Google Colab y adjuntos a este informe:

- Lab4\_51\_GG1\_160005038.ipynb (sección 5.1),
- Lab4\_52\_RedColas\_160005038.ipynb (sección 5.2),



- Lab4\_53\_Inventario\_160005038.ipynb (sección 5.3),
- Lab4\_54\_SimPy\_160005038.ipynb (sección 5.4).

Los modelos de la sección 5.5 se construyeron en AnyLogic PLE y se publicaron en AnyLogic Cloud.

### 3.1. Generador de números pseudoaleatorios y variables aleatorias

Como en los laboratorios anteriores, no se usó ninguna función de aleatoriedad de librería: todos los números aleatorios provienen del generador congruencial mixto GLC-d del Laboratorio 2,  $X_{i+1} = (25214903917 X_i + 11) \bmod 2^{48}$ , que cumple las condiciones de Hull–Dobell [5] y cuya uniformidad y aleatoriedad ya fueron verificadas. Sobre él se usan los generadores del laboratorio anterior: exponencial por transformada inversa,  $X = -\frac{1}{\lambda} \ln(1 - U)$ , y normal por el método polar de Marsaglia [6], que produce dos normales por par aceptado (la segunda se guarda para la siguiente llamada). La guía exige truncar en cero las normales negativas, lo que se implementa en `normal_truncada`. Cada escenario y cada repetición usan semillas distintas para trabajar con flujos independientes.

```
1 A, C, M = 25214903917, 11, 2**48
2
3 class GLC:
4     def __init__(self, X0, a=A, c=C, m=M):
5         self.X, self.a, self.c, self.m = X0, a, c, m
6         self.consumidos = 0
7     def uniforme(self):
8         self.X = (self.a * self.X + self.c) % self.m
9         self.consumidos += 1
10        return self.X / self.m
11
12 def exponencial(g, lam): # lam = tasa
13     return -math.log(1 - g.uniforme()) / lam
14
15 _cache = {}
16 def normal_polar(g, mu=0.0, sigma=1.0): # metodo polar; guarda la 2a normal del
17     par
18     if g in _cache:
19         z = _cache.pop(g)
20     else:
21         while True:
22             v1, v2 = 2 * g.uniforme() - 1, 2 * g.uniforme() - 1
23             s = v1 * v1 + v2 * v2
24             if 0 < s <= 1:
25                 k = math.sqrt(-2 * math.log(s) / s)
26                 z, _cache[g] = v1 * k, v2 * k
27                 break
28     return mu + sigma * z
29
30 def normal_truncada(g, mu, sigma): # valores negativos -> 0 (nota de la
31     guia)
32     return max(0.0, normal_polar(g, mu, sigma))
```

**Bloque de código 1:** Generador congruencial mixto y generadores de variables aleatorias usados en todo el laboratorio.



### 3.2. Sistema de línea de espera con un servidor (5.1)

Se partió del notebook del curso *Sistema de línea de espera con un servidor - G/G/1.ipynb*, que implementa el algoritmo de la Sec. 5.5.1 de [2] con dos rutinas, *Llegada* y *Servidor*. Se hicieron tres modificaciones: (i) los dos generadores congruenciales del ejemplo se reemplazaron por dos flujos del GLC (uno para llegadas y otro para servicios); (ii) el programa principal se encapsuló en la función `simular_gg1(T, genX, genY)`, que recibe como argumentos los generadores de tiempos entre llegadas y de servicio y devuelve las cinco medidas de desempeño junto con las trayectorias para graficar; y (iii) se añadió la función `repetir` para ejecutar  $n$  réplicas con semillas distintas y calcular promedio y desviación estándar muestral. Con esta estructura los dos escenarios se obtienen cambiando únicamente los argumentos:

- **Escenario A:** `genX = exponencial(g, 5)`, `genY = exponencial(g, 3)`,  $T = 40$ . El parámetro  $\lambda$  es la tasa, como en el notebook base ( $X = -\ln(1 - U)/\lambda$ ).
- **Escenario B:** `genX = exponencial(g, 1)`, `genY = normal_truncada(g, 2, 1)`,  $T = 40$ .

```
1 def simular_gg1(T, genX, genY, M=99999.0):
2     est = dict(t=0.0, n=0, NLL=0, NS=0)
3     TSuc = {'tLL': M, 'tS': M}
4     LL, S, Serv = [], [], []          # llegada, salida y servicio de cada
        cliente
5
6     def Llegada(tsuc):
7         est['t'] = tsuc; est['n'] += 1; est['NLL'] += 1; LL.append(tsuc)
8         X = genX()
9         if est['t'] + X < T:           # solo se admiten llegadas antes de T
10             TSuc['tLL'] = est['t'] + X
11         if est['n'] == 1:             # servidor libre: empieza su servicio
12             Y = genY(); TSuc['tS'] = est['t'] + Y; Serv.append(Y)
13
14     def Servidor(tsuc):
15         est['t'] = tsuc; est['n'] -= 1; est['NS'] += 1; S.append(tsuc)
16         if est['n'] > 0:               # el siguiente de la cola pasa a servicio
17             Y = genY(); TSuc['tS'] = est['t'] + Y; Serv.append(Y)
18
19     X = genX()
20     if X > T:
21         return medidas_nulas()
22     Llegada(X)
23     while TSuc['tLL'] != M or TSuc['tS'] != M:
24         if TSuc['tLL'] < TSuc['tS']:
25             tsuc = TSuc['tLL']; TSuc['tLL'] = M; Llegada(tsuc)
26         else:
27             tsuc = TSuc['tS']; TSuc['tS'] = M; Servidor(tsuc)
28     Tp = max(0.0, est['t'] - T)
29     t_med_sistema = sum(S[i] - LL[i] for i in range(est['NLL'])) / est['NLL']
30     t_med_cola = sum(S[i] - LL[i] - Serv[i] for i in range(est['NLL'])) / est['NLL']
31     ...
```

Bloque de código 2: Núcleo del modelo  $G/G/1$ : rutinas de suceso y bucle principal.

### 3.3. Red de colas (5.2)

Se partió del notebook del curso *Red de Colas.ipynb*, que implementa las cuatro rutinas de la Sec. 5.5.2 de [2]. Se conservó la lógica de cada rutina y se hicieron los siguientes cambios: (i) las



llamadas a `numpy.random` se reemplazaron por el GLC y los generadores propios; (ii) el bucle principal, que en el ejemplo evalúa cuatro `if` consecutivos recalculando el mínimo cada vez, se reescribió extrayendo el suceso de menor instante con `min(TSuc, key=TSuc.get)` y despachándolo mediante un diccionario de rutinas, lo que garantiza que en cada iteración se procesa exactamente un suceso; (iii) todo se encapsuló en `simular_red(p, g)`, con los parámetros en un diccionario, para poder repetirla.

**Convención de parámetros.** El notebook base genera los tiempos exponenciales con la función `np.random.exponential(L)`, cuyo argumento es la *media* (escala) de la distribución, no la tasa. Para que los escenarios sean comparables con el ejemplo, los parámetros  $\lambda$  y  $\mu_2$  de la guía se interpretan de la misma forma: media de la exponencial. Con esta convención la tasa de llegadas es  $1/\lambda$  y la intensidad de tráfico de cada nodo  $j$  es  $\rho_j = \lambda_j \bar{s}_j$ , con  $\lambda_j$  la tasa de entrada al nodo y  $\bar{s}_j$  su tiempo medio de servicio, lo que se usa en el análisis. La Tabla 1 resume los dos escenarios.

**Tabla 1:** Parámetros de los dos escenarios de la red de colas ( $T = 300$ ).

| Parámetro                                                     | Escenario A | Escenario B |
|---------------------------------------------------------------|-------------|-------------|
| Llegadas: $\mathcal{Exp}$ de media $\lambda$                  | 2           | 3           |
| Nodo 1: $\mathcal{N}(\mu_1, \sigma_1)$                        | (2,1; 0,5)  | (3,0; 1,0)  |
| Nodo 2: $\mathcal{Exp}$ de media $\mu_2$                      | 2,4         | 1,5         |
| Nodo 3 si $n_3 < 5$ : $\mathcal{N}(\mu_{31}, \sigma_{31})$    | (4,1; 2,5)  | (4,0; 2,0)  |
| Nodo 3 si $n_3 \geq 5$ : $\mathcal{N}(\mu_{32}, \sigma_{32})$ | (3,0; 2,0)  | (2,0; 1,5)  |

```
1 def Servicio_nodo1(tsuc):
2     e['n_med_n1'] += e['n1'] * (tsuc - e['t']); e['n1'] -= 1
3     e['NS1'] += 1; S1.append(tsuc)
4     if g.uniforme() <= 0.4: # pruebas adicionales (nodo 2)
5         e['n_med_n2'] += e['n2'] * (tsuc - e['t']); e['n2'] += 1
6         e['NLL2'] += 1; LL2.append(tsuc)
7         if e['n2'] == 1:
8             TSuc['tS2'] = tsuc + exponencial_media(g, p['mu2'])
9     else: # directo a pruebas estandar (
10         nodo 3)
11         e['n_med_n3'] += e['n3'] * (tsuc - e['t']); e['n3'] += 1
12         e['NLL3'] += 1; LL3.append(tsuc)
13         if e['n3'] == 1:
14             TSuc['tS3'] = tsuc + servicio_n3() # N(mu31, s31) si n3<5, N(mu32,
15                 s32) si no
16         e['t'] = tsuc
17         if e['n1'] > 0:
18             TSuc['tS1'] = tsuc + normal_truncada(g, p['mu1'], p['sigma1'])
19
20 rutinas = {"tLL1": Llegada_cliente, "tS1": Servicio_nodo1,
21           "tS2": Servicio_nodo2, "tS3": Servicio_nodo3}
22 while min(TSuc.values()) != M:
23     clave = min(TSuc, key=TSuc.get) # proximo suceso
24     tsuc = TSuc[clave]; TSuc[clave] = M
25     rutinas[clave](tsuc)
```

**Bloque de código 3:** Red de colas: rutina del nodo 1 (con bifurcación 0,4/0,6) y bucle principal con despacho por diccionario.

Para las 20 repeticiones de cada escenario se guardaron las diez medidas de cada réplica y se calcularon la media muestral  $\bar{Y}$ , la desviación estándar muestral  $S$  ( $n - 1$  en el denominador) y el



intervalo de confianza del 95 % basado en la distribución  $t$  de Student con  $n - 1 = 19$  grados de libertad,

$$\bar{Y} \pm t_{0,975;19} \frac{S}{\sqrt{n}}, \quad t_{0,975;19} = 2,093, \quad (1)$$

que es el procedimiento del Capítulo 8 de [2] para réplicas independientes.

### 3.4. Inventario probabilístico (5.3)

Siguiendo la misma estructura de los dos modelos anteriores se implementó la función `simular_inventario` con dos sucesos, *llegada de un cliente* ( $t_C$ ) y *llegada de un pedido* ( $t_P$ ), y la lista  $TSuc = \{t_C, t_P\}$ . El estado está formado por el nivel de inventario  $x$  y la cantidad en pedido pendiente  $y$ ; los acumuladores son el costo de almacenamiento (que se cobra por el tiempo que  $x$  se mantuvo constante entre sucesos), el costo de pedidos, los ingresos y el tiempo con  $x = 0$ . La política es la de la Figura 1 de la guía: cuando tras una venta  $x < p$  y  $y = 0$ , se pide  $y = P - x$  unidades a costo  $h$  y se programa  $t_P = t + L$ . Si la demanda  $D$  de un cliente supera el inventario se le venden las  $x$  unidades disponibles y el cliente se cuenta como no satisfecho.

**Parámetros.** La guía fija  $P_0 = 20$ ,  $P = 100$ ,  $p = 20$ ,  $L = 5$ ,  $h = 100$  y  $r = 150$ . No fija el parámetro  $\lambda$  del proceso de Poisson de llegadas, la distribución de la cantidad demandada por cada cliente, el costo de almacenamiento por unidad y tiempo ni el horizonte; se adoptaron  $\lambda = 2$  clientes por unidad de tiempo,  $D \sim$  Uniforme discreta  $\{1, \dots, 5\}$  (media 3),  $c_a = 1$  y  $T = 100$ , y se dejaron como parámetros del notebook para poder cambiarlos.<sup>1</sup>

```
1 TSuc = {'tC': exponencial(g, lam), 'tP': M}
2 while min(TSuc.values()) <= T:
3     if TSuc['tC'] <= TSuc['tP']:                                # ---- llegada de un cliente
4         # ----
5         avanzar(TSuc['tC'])                                     # acumula ca*x*(dt) y tiempo
6         con x=0
7         D = demanda(); clientes += 1
8         vendido = min(D, x)
9         if vendido == D: satisfechos += 1
10        x -= vendido; ingresos += r * vendido
11        if x < p and y == 0:                                     # punto de nuevo pedido
12            y = P - x; C_ped += h * y; TSuc['tP'] = t + L
13            TSuc['tC'] = t + exponencial(g, lam)
14        else:                                                    # ---- llegada del pedido ----
15            avanzar(TSuc['tP']); x += y; y = 0; TSuc['tP'] = M
16        registrar()                                             # trayectorias para las
17        graficas
18 beneficio = ingresos - C_ped - C_alm
```

**Bloque de código 4:** Inventario probabilístico: bucle de sucesos y política ( $p, P$ ).

Además de la corrida única que pide la guía se hicieron 20 repeticiones y un análisis de sensibilidad del punto de pedido  $p \in \{10, 20, \dots, 60\}$  para interpretar los resultados.

<sup>1</sup>Con estos supuestos la demanda media es de  $\lambda E[D] = 6$  unidades por unidad de tiempo, de modo que durante el plazo de entrega  $L = 5$  se demandan en promedio 30 unidades, más que el punto de pedido  $p = 20$ : el modelo se configura deliberadamente con riesgo de desabastecimiento, como sugiere la Figura 1 de la guía.



### 3.5. Simulación de eventos discretos en SimPy (5.4)

Se instaló SimPy 4.1.2 con pip y se recorrió el tutorial *SimPy in 10 minutes* [3]: un proceso básico (carro que alterna carga y conducción con `env.timeout()`), interacción entre procesos (`yield env.process(...)` e `Interrupt`) y recursos compartidos (estación de carga con `Resource(capacity=2)`). Después se implementaron versiones propias, comentadas en español, de los siete ejemplos de la documentación: *Bank Renege*, *Carwash*, *Machine Shop*, *Movie Renege*, *Gas Station Refueling*, *Process Communication* y *Event Latency*. Para mantener la coherencia con el resto del laboratorio, la aleatoriedad de todos los ejemplos proviene del GLC, inyectado en una subclase de `random.Random` que redefine `random()` y `getrandbits()`; así `expovariate`, `uniform`, `randint` y `choice` consumen uniformes del generador congruencial.

```
1 class GLCRandom(random.Random):
2     def __init__(self, X0):
3         super().__init__(0); self.X = X0
4     def random(self):
5         self.X = (A * self.X + C) % M_GLC
6         return self.X / M_GLC
7     def getrandbits(self, k):
8         return int(self.random() * (1 << k))
9
10 def cliente(env, nombre):
11     llegada = env.now
12     with cajero.request() as req:
13         pac = rng.uniform(1, 3)                # paciencia
14         res = yield req | env.timeout(pac)      # lo que ocurra primero
15         if req in res:                          # le toca el turno
16             yield env.timeout(rng.expovariate(1.0 / 12.0))
17         else:                                  # se agota la paciencia:
18             abandona
19             abandonos += 1
```

**Bloque de código 5:** Fuente de aleatoriedad para SimPy basada en el GLC, y ejemplo *Bank Renege* (espera con abandono).

En *Gas Station Refueling* se introdujo una mejora respecto al ejemplo original: el proceso de control llama al camión cisterna no solo cuando el nivel del depósito baja del 10 %, sino también cuando hay carros esperando combustible (`deposito.get_queue`), porque con el umbral solo el surtidor puede bloquearse si un carro pide más litros de los que quedan sin que el nivel haya cruzado el umbral.

### 3.6. Simulación de eventos discretos en AnyLogic (5.5)

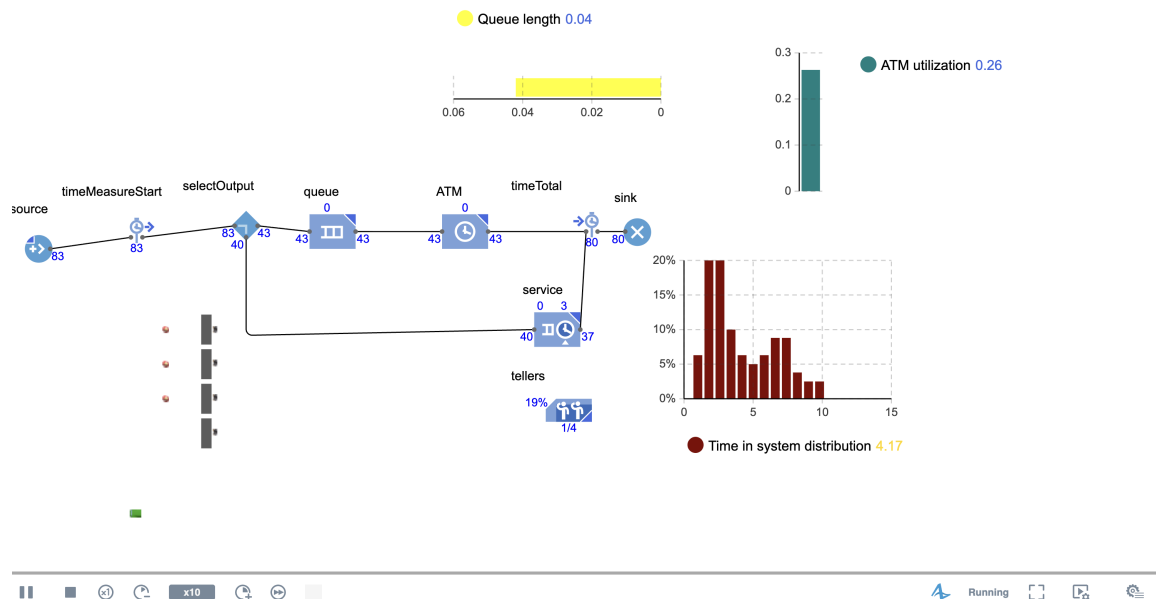
Se instaló AnyLogic PLE y se desarrollaron paso a paso los cinco tutoriales de la guía [4]. Los modelos resultantes (*Bank Office*, *Subway entrance*, *Road Traffic*, *Job Shop* y *Lead Acid Battery Production*, cada uno con su archivo `.alp` y sus recursos 3D) se subieron a AnyLogic Cloud. A continuación se describe la construcción de cada modelo y se muestran las capturas del diagrama de proceso en el editor y de la simulación en ejecución.

#### 3.6.1. Bank Office (Process Modeling Library)

Se modeló una oficina bancaria con un cajero automático y cuatro cajeros humanos. Pasos: (1) crear el modelo y dibujar el plano con los nodos de espera y las rutas; (2) construir el diagrama de proceso `source` (llegadas de clientes) → `timeMeasureStart` → `selectOutput` (bifurcación: la mitad va al cajero automático y la otra mitad a ventanilla) → `queue` → `ATM` (bloque `Delay`)



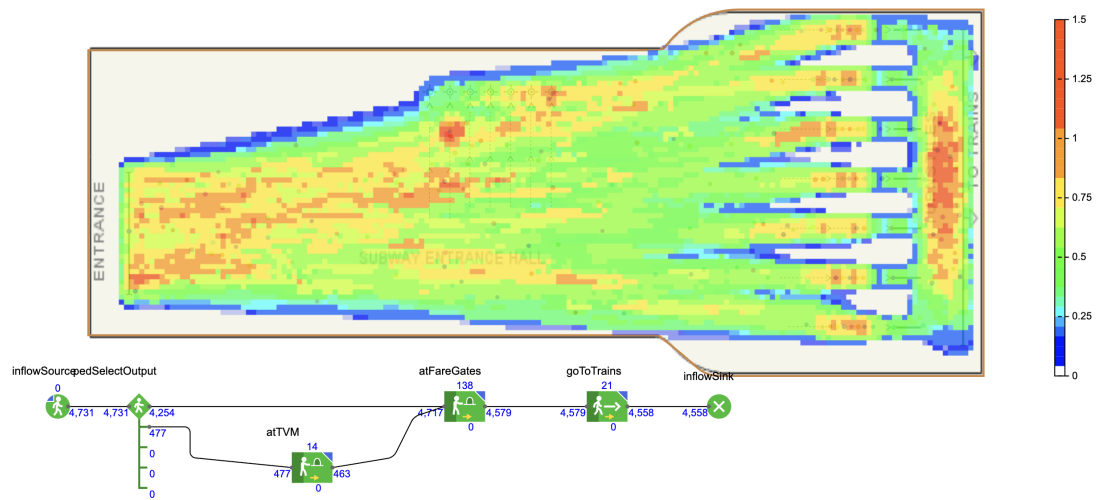
y, por la otra rama, **service** con el **ResourcePool** **tellers** de cuatro cajeros → **timeTotal** (**TimeMeasureEnd**) → **sink**; (3) asociar cada bloque a su elemento del plano y añadir la animación 2D de clientes y cajeros; (4) agregar las estadísticas: longitud media de la cola del cajero automático, utilización del cajero automático y de los **tellers**, e histograma del tiempo en el sistema. La Figura 1 muestra el modelo en ejecución.



**Figura 1:** Bank Office en ejecución: diagrama de proceso con los contadores de cada bloque (83 clientes llegados, 43 al cajero automático y 40 a ventanilla), longitud media de la cola, utilización del cajero automático y de los cajeros, e histograma del tiempo en el sistema.

### 3.6.2. Subway entrance (Pedestrian Library)

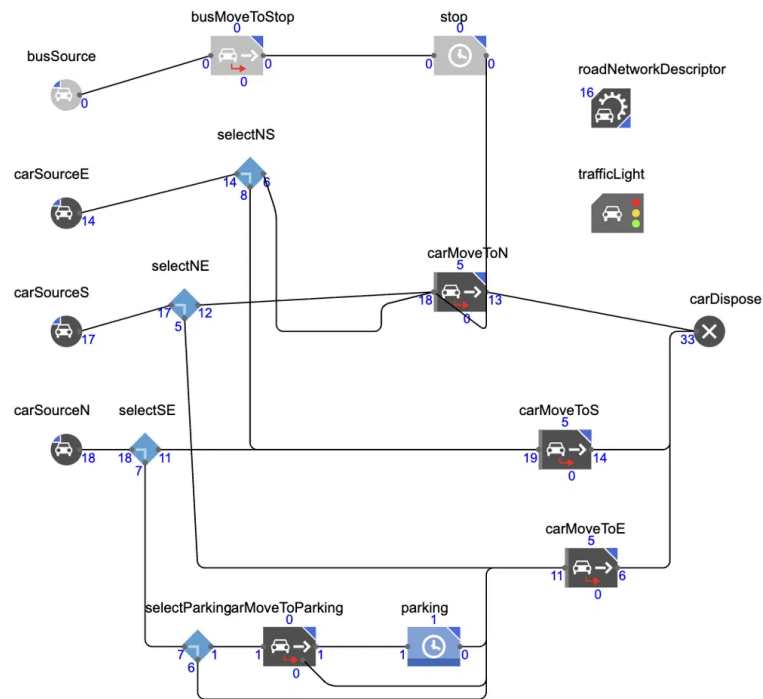
Se modeló el flujo de pasajeros en el vestíbulo de una estación de metro. Pasos: (1) importar el plano **hall\_drawing.png**, dibujar las paredes con **Wall** y definir las **Target Line** de entrada (**ENTRANCE**) y de salida hacia los trenes; (2) ubicar las máquinas expendedoras (TVM) y los torniquetes como servicios con fila (**Service with Lines**); (3) construir el diagrama **inflowSource** → **pedSelectOutput** (el 10 % compra boleto) → **atTVM** → **atFareGates** → **goToTrains** → **inflowSink**, con la rama directa para quienes ya tienen boleto; (4) añadir el mapa de densidad (**Density Map**) que colorea el plano según la densidad de peatones por  $m^2$ . La Figura 2 muestra el modelo en ejecución.



**Figura 2:** Subway entrance en ejecución: mapa de densidad de peatones sobre el plano del vestíbulo (escala 0–1,5 peatones/m<sup>2</sup>) y diagrama de proceso peatonal (4731 pasajeros generados, 477 pasan por las máquinas expendedoras).

### 3.6.3. Intersection (Road Traffic Library)

Se modeló una intersección en T con semáforo, parada de bus y una zona de estacionamiento. Pasos: (1) importar la imagen satelital `layout.jpg` y dibujar las tres vías con `Road` y la intersección con `Intersection`, definiendo los giros permitidos; (2) crear los tipos de agente `Car` y `Bus` y los generadores `carSourceN`, `carSourceS`, `carSourceE` y `busSource`; (3) construir el diagrama: cada fuente pasa por un `SelectOutput` (`selectNS`, `selectNE`, `selectSE`) que elige el destino y por el `CarMoveTo` correspondiente (`carMoveToN`, `carMoveToS`, `carMoveToE`) hasta `carDispose`; el bus hace `busMoveToStop` → `stop` (`Delay`) y una fracción de los carros del norte entra a `parking`; (4) añadir el `trafficLight` con sus fases y el `roadNetworkDescriptor`, que colorea cada vía según su densidad de tráfico. Las Figuras 3 y 4 muestran el diagrama y la simulación.



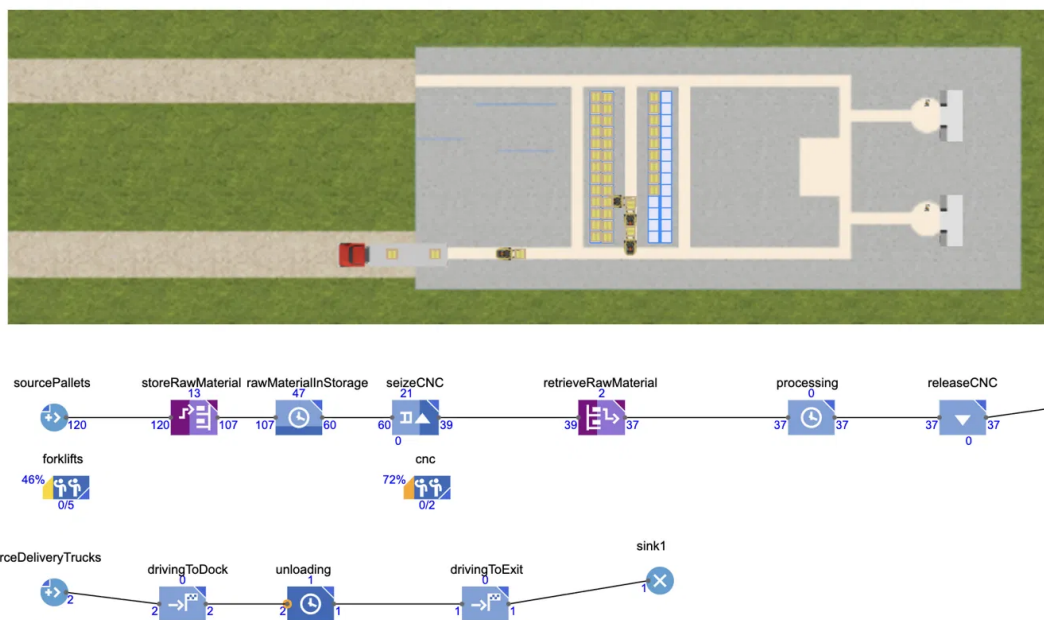
**Figura 3:** Intersection: diagrama de proceso de la Road Traffic Library con las tres fuentes de carros, la fuente de buses, los bloques de selección de destino, los CarMoveTo, el estacionamiento y el semáforo.



**Figura 4:** Intersection en ejecución: vehículos circulando sobre la imagen satelital; el roadNetworkDescriptor colorea las vías por densidad de tráfico (verde baja, rojo alta en la aproximación al semáforo).

### 3.6.4. Job Shop (Process Modeling Library con recursos móviles)

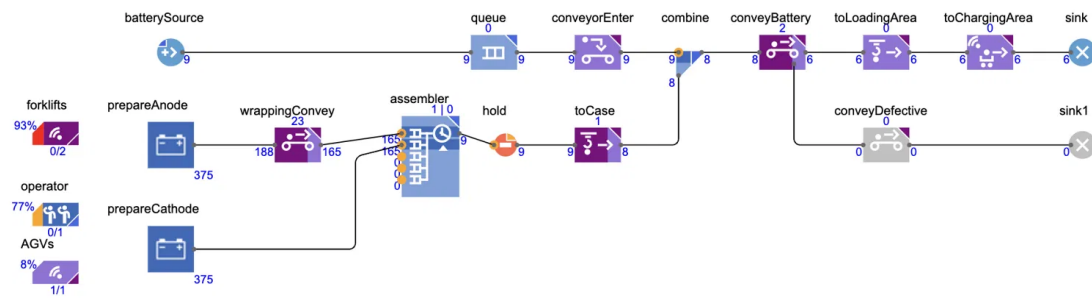
Se modeló un taller en el que llegan camiones con paletas de materia prima, los montacargas las almacenan en estanterías, luego las llevan a dos centros de mecanizado CNC y las piezas terminadas salen del sistema. Pasos: (1) importar el plano y dibujar los nodos (muelle de descarga, almacén con Pallet Rack, área CNC) y las rutas de los montacargas; (2) definir los recursos **forklifts** (5 montacargas, móviles) y **cnc** (2 máquinas); (3) construir el flujo de paletas **sourcePallets** → **storeRawMaterial** (**RackStore**) → **rawMaterialInStorage** (**Delay**) → **seizeCNC** → **retrieveRawMaterial** (**RackPick**, con montacargas) → **processing** → **releaseCNC** → **sink**, y el flujo de camiones **sourceDeliveryTrucks** → **drivingToDock** → **unloading** → **drivingToExit** → **sink1**; (4) añadir la animación con los modelos 3D del camión, los montacargas, las paletas y las máquinas CNC. La Figura 5 muestra el modelo en ejecución.



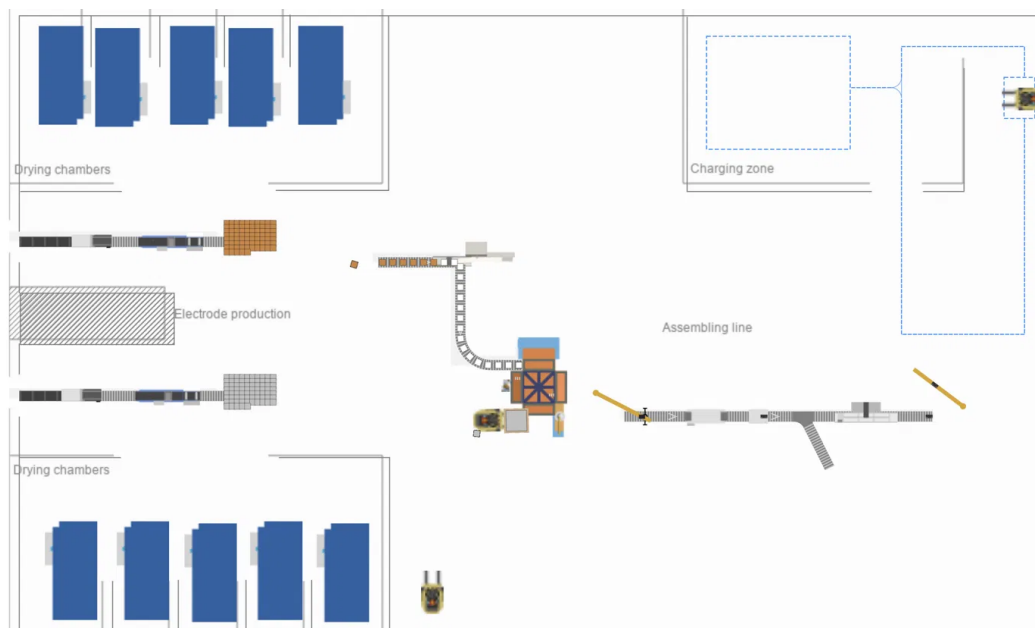
**Figura 5:** Job Shop en ejecución: plano del taller con el camión descargando y los montacargas en el almacén, y los dos diagramas de proceso (paletas y camiones) con la utilización de los recursos (**forklifts** 46 %, **cnc** 72 %).

### 3.6.5. Lead Acid Battery Production (Material Handling Library)

Se modeló una planta de producción de baterías con transportadores, montacargas y un vehículo de guiado automático (AGV). Pasos: (1) dibujar el plano con las zonas de producción de electrodos, las cámaras de secado, la línea de ensamble y la zona de carga; (2) definir los tipos de agente (ánodo, cátodo, batería) y los recursos **forklifts**, **operator** y **AGVs**; (3) construir el diagrama: dos fuentes **prepareAnode** y **prepareCathode** alimentan, a través de **wrappingConvey**, el bloque **assembler** que ensambla las placas; la batería pasa por **hold** y **toCase**, se combina con la carcasa que llega por **batterySource** → **queue** → **conveyorEnter** en el bloque **combine**, y sigue por **conveyBattery** → **toLoadingArea** → **toChargingArea** (transporte con AGV) → **sink**; las defectuosas se desvían por **conveyDefective** → **sink1**; (4) dibujar la red de transportadores (**Conveyor**, **Station**, **Turntable**) y la ruta del AGV, y añadir la animación con los modelos 3D de cada máquina. Las Figuras 6 y 7 muestran el diagrama y la planta en ejecución.



**Figura 6:** Lead Acid Battery Production: diagrama de proceso de la Material Handling Library con los flujos de electrodos, carcasas y baterías, y la utilización de los recursos (forklifts 93 %, operator 77 %, AGVs 8 %).



**Figura 7:** Lead Acid Battery Production en ejecución: plano de la planta con las cámaras de secado, la producción de electrodos, la línea de ensamble con transportadores, la zona de carga y el AGV.

## 4. Resultados y análisis

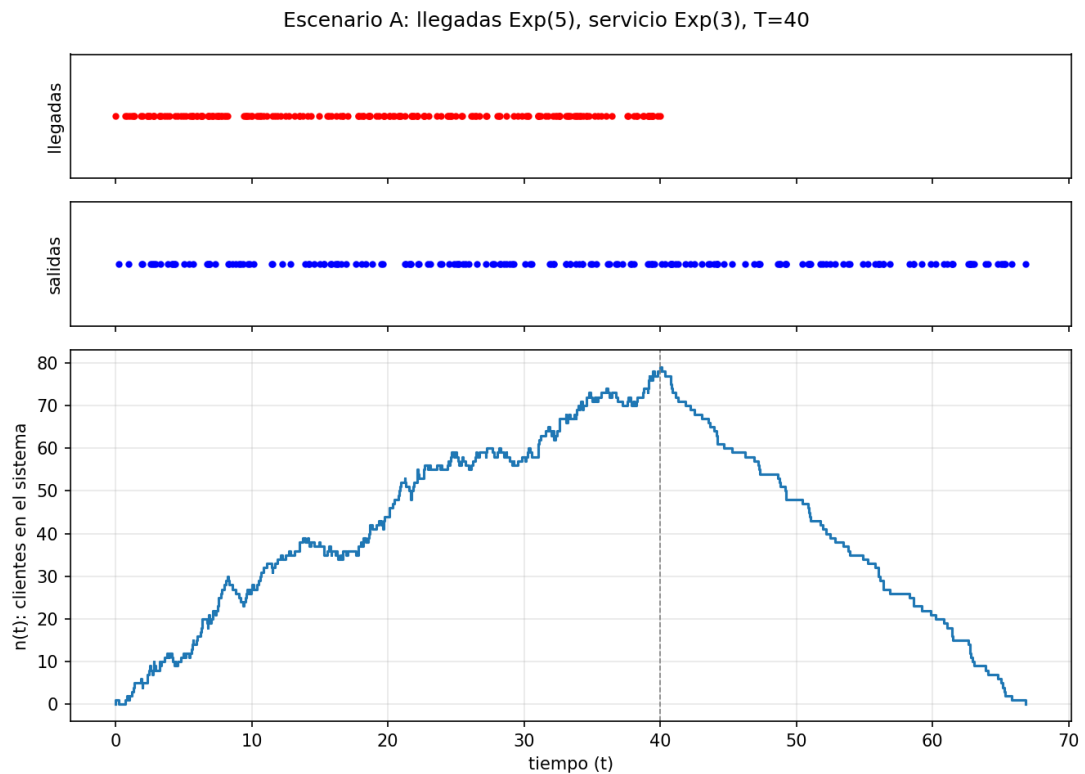
### 4.1. Sistema de línea de espera con un servidor (5.1)

La Tabla 2 muestra las medidas de desempeño de una ejecución de cada escenario y las Figuras 8 y 9 sus trayectorias  $n(t)$  con los instantes de llegada y de salida.

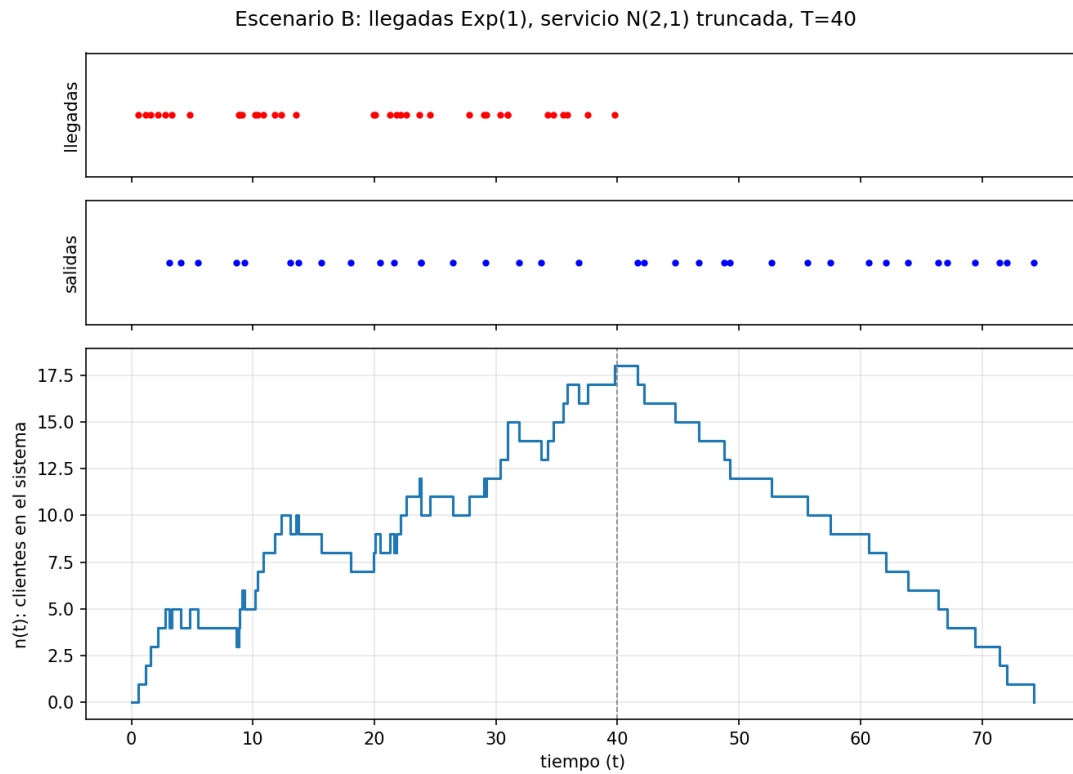


**Tabla 2:** Medidas de desempeño de una ejecución de cada escenario del  $G/G/1$  ( $T = 40$ ).

| Medida de desempeño                                                        | Escenario A | Escenario B |
|----------------------------------------------------------------------------|-------------|-------------|
| Tiempo medio de los clientes en el sistema                                 | 14,107      | 19,472      |
| Tiempo medio de los clientes en la cola                                    | 13,769      | 17,425      |
| Tiempo desde $T$ hasta que el último cliente abandona el sistema ( $T_p$ ) | 26,842      | 34,262      |
| Número máximo de clientes en el sistema                                    | 79          | 18          |
| Total de clientes que pasaron por el sistema                               | 196         | 36          |



**Figura 8:** Escenario A ( $\mathcal{Exp}(5)/\mathcal{Exp}(3)$ ): instantes de llegada (rojo), de salida (azul) y número de clientes en el sistema  $n(t)$ . La línea discontinua marca  $T = 40$ .

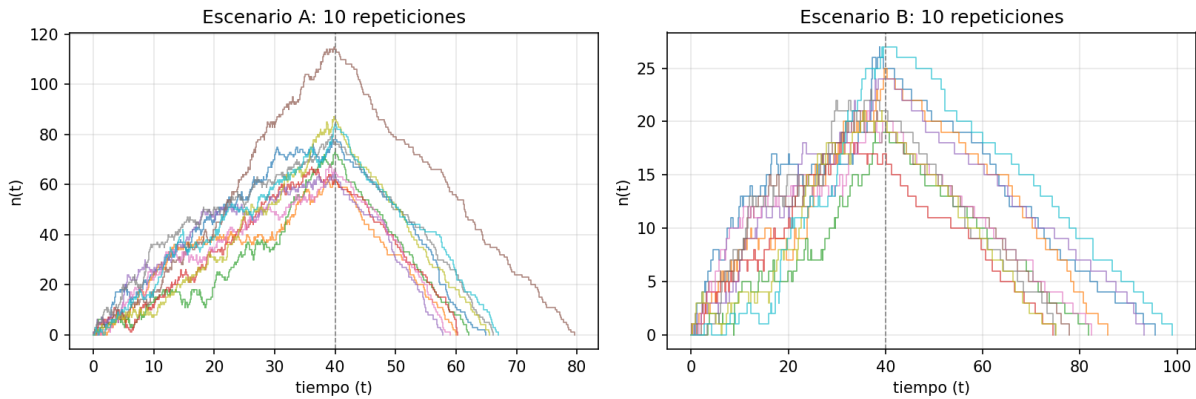


**Figura 9:** Escenario B ( $\mathcal{Exp}(1)/\mathcal{N}(2,1)$  truncada): instantes de llegada, de salida y  $n(t)$ .

La Tabla 3 corresponde a la Tabla 1 de la guía: promedio y desviación estándar muestral de las cinco medidas en 10 repeticiones independientes de cada escenario; la Figura 10 superpone las 10 trayectorias.

**Tabla 3:** Medidas de desempeño para los escenarios A y B luego de 10 simulaciones (promedio  $\pm$  desviación estándar).

| Medida de desempeño                                              | Escenario A        | Escenario B        |
|------------------------------------------------------------------|--------------------|--------------------|
| Tiempo medio de los clientes en el sistema                       | $12,757 \pm 2,971$ | $23,847 \pm 3,745$ |
| Tiempo medio de los clientes en la cola                          | $12,435 \pm 2,953$ | $21,794 \pm 3,664$ |
| Tiempo desde $T$ hasta que el último cliente abandona el sistema | $24,237 \pm 6,279$ | $43,988 \pm 9,064$ |
| Número máximo de clientes en el sistema                          | $78,0 \pm 15,5$    | $22,6 \pm 3,1$     |
| Total de clientes que pasaron por el sistema                     | $198,1 \pm 9,6$    | $40,1 \pm 3,9$     |



**Figura 10:** Trayectorias  $n(t)$  de las 10 repeticiones de cada escenario.

**Análisis.** Los dos escenarios están *saturados*. En A la tasa de llegadas ( $\lambda = 5$ ) es mayor que la de servicio ( $\mu = 3$ ),  $\rho = 5/3 \approx 1,67$ ; en B llega un cliente por unidad de tiempo y cada servicio dura en promedio  $E[\max(0, \mathcal{N}(2, 1))] \approx 2,008$ , de modo que  $\rho \approx 2$ . Por eso en ambos casos  $n(t)$  crece de forma casi lineal hasta  $T$  (Figura 10) y solo decrece cuando se cierran las entradas; el sistema tarda  $T_p \approx 24$  y  $\approx 44$  unidades adicionales en vaciarse, es decir, entre el 60 % y el 110 % del horizonte. En un sistema saturado las medidas temporales no convergen a un valor estacionario sino que dependen de  $T$ : los tiempos medios reportados deben leerse como propios de este horizonte.

La comparación entre escenarios es consistente con sus parámetros. A recibe cinco veces más clientes que B (198 frente a 40) y acumula colas mucho más largas (máximo 78 frente a 23), pero cada cliente espera menos (12,8 frente a 23,8) porque el servidor también es más rápido: la cola de A se consume a tasa 3 y la de B a tasa 0,5. En términos relativos la pendiente de crecimiento de la cola,  $\lambda - \mu$ , es 2 clientes por unidad de tiempo en A y 0,5 en B, lo que coincide con los máximos observados ( $\approx 2 \times 40 = 80$  y  $\approx 0,5 \times 40 = 20$ ). La fracción del tiempo en el sistema que se pasa en cola es del 97,5 % en A y del 91 % en B, reflejo de que el servicio es una parte minúscula de la permanencia cuando  $\rho > 1$ . En cuanto a la variabilidad, los coeficientes de variación de los tiempos medios son del 23 % en A y del 16 % en B: la normal truncada con  $\sigma/\mu = 0,5$  es menos dispersa que la exponencial ( $\sigma/\mu = 1$ ), y aunque B tiene menos clientes por réplica, su tiempo de servicio más regular compensa parcialmente la menor muestra. Finalmente, diez repeticiones bastan para estimar las medias con errores estándar del 7 % al 10 %, pero no para distinguir detalles finos de la distribución de  $T_p$ , cuya desviación estándar es del orden del 25 % de la media.

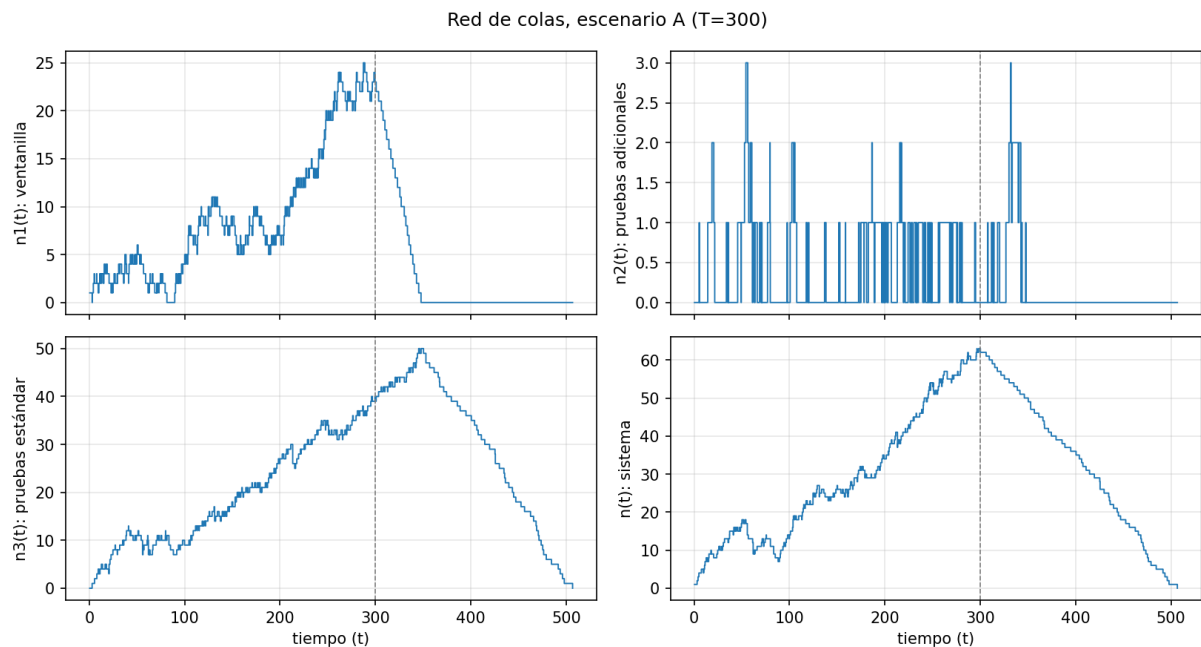
#### 4.2. Red de colas (5.2)

La Tabla 4 muestra las diez medidas de una ejecución de cada escenario y las Figuras 11 y 12 las trayectorias  $n_1(t)$ ,  $n_2(t)$ ,  $n_3(t)$  y  $n(t)$ .

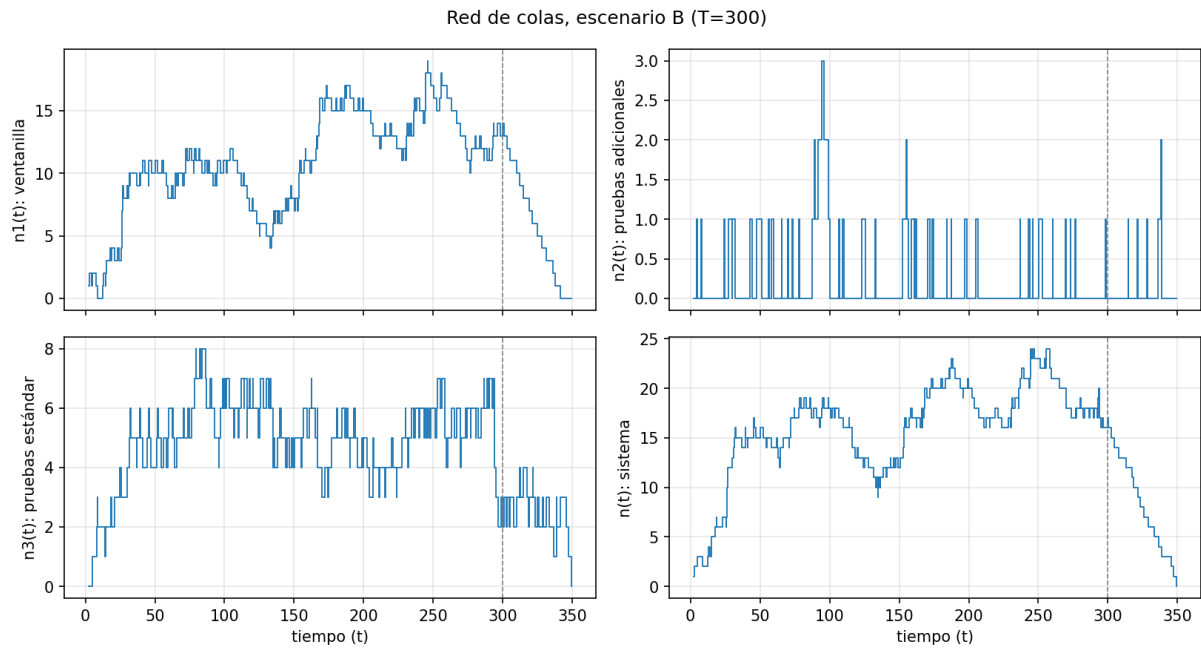


**Tabla 4:** Medidas de desempeño de una ejecución de cada escenario de la red de colas ( $T = 300$ ).

| Medida de desempeño                                                        | Escenario A | Escenario B |
|----------------------------------------------------------------------------|-------------|-------------|
| Tiempo medio de los clientes en el sistema ( $t_{\text{med\_sistema}}$ )   | 94,216      | 46,381      |
| Número promedio de clientes en el nodo 1 ( $n_{\text{med\_n1}}$ )          | 4,570       | 6,372       |
| Número promedio de clientes en el nodo 2 ( $n_{\text{med\_n2}}$ )          | 0,128       | 0,084       |
| Número promedio de clientes en el nodo 3 ( $n_{\text{med\_n3}}$ )          | 16,580      | 2,599       |
| Tiempo desde $T$ hasta que el último cliente abandona el sistema ( $T_p$ ) | 206,726     | 49,452      |
| Número máximo de clientes en el sistema                                    | 63          | 24          |
| Total de clientes que pasaron por el nodo 1                                | 163         | 112         |
| Total de clientes que pasaron por el nodo 2                                | 67          | 45          |
| Total de clientes que pasaron por el nodo 3                                | 163         | 112         |
| Total de clientes que pasaron por el sistema                               | 163         | 112         |



**Figura 11:** Escenario A de la red de colas: número de clientes en cada nodo y en el sistema.



**Figura 12:** Escenario B de la red de colas: número de clientes en cada nodo y en el sistema.

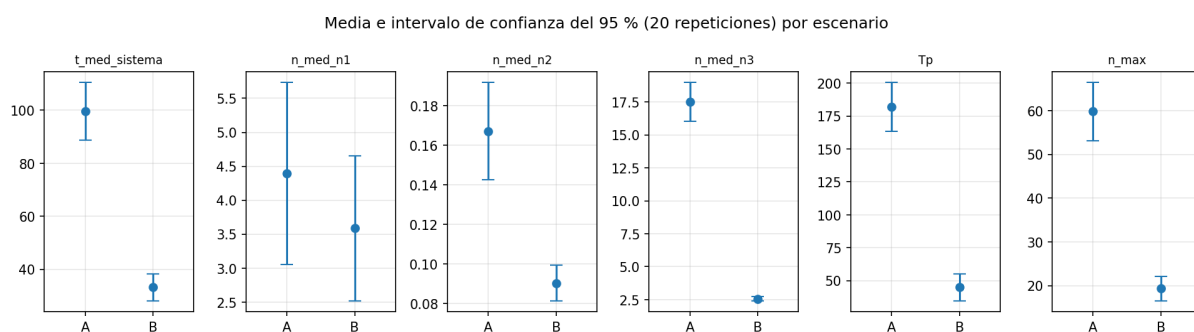
Las Tablas 5 y 6 resumen las 20 repeticiones de cada escenario con la media, la desviación estándar muestral y el intervalo de confianza del 95 % de la ecuación (1); la Figura 13 compara gráficamente los intervalos de las seis primeras medidas.

**Tabla 5:** Escenario A: media, desviación estándar e intervalo de confianza del 95 % ( $n = 20$  repeticiones).

| Medida de desempeño                                         | Media   | Desv. est. | IC 95 %            |
|-------------------------------------------------------------|---------|------------|--------------------|
| Tiempo medio de los clientes en el sistema                  | 99,536  | 23,329     | [88,618; 110,454]  |
| Número promedio de clientes en el nodo 1                    | 4,391   | 2,864      | [3,051; 5,732]     |
| Número promedio de clientes en el nodo 2                    | 0,167   | 0,052      | [0,143; 0,192]     |
| Número promedio de clientes en el nodo 3                    | 17,519  | 3,119      | [16,059; 18,978]   |
| Tiempo desde $T$ hasta que sale el último cliente ( $T_p$ ) | 181,939 | 39,345     | [163,526; 200,353] |
| Número máximo de clientes en el sistema                     | 59,800  | 14,126     | [53,189; 66,411]   |
| Total de clientes que pasaron por el nodo 1                 | 151,650 | 13,903     | [145,143; 158,157] |
| Total de clientes que pasaron por el nodo 2                 | 60,350  | 6,923      | [57,110; 63,590]   |
| Total de clientes que pasaron por el nodo 3                 | 151,650 | 13,903     | [145,143; 158,157] |
| Total de clientes que pasaron por el sistema                | 151,650 | 13,903     | [145,143; 158,157] |

**Tabla 6:** Escenario B: media, desviación estándar e intervalo de confianza del 95 % ( $n = 20$  repeticiones).

| Medida de desempeño                                         | Media   | Desv. est. | IC 95 %           |
|-------------------------------------------------------------|---------|------------|-------------------|
| Tiempo medio de los clientes en el sistema                  | 33,150  | 10,928     | [28,035; 38,264]  |
| Número promedio de clientes en el nodo 1                    | 3,586   | 2,282      | [2,518; 4,654]    |
| Número promedio de clientes en el nodo 2                    | 0,090   | 0,019      | [0,081; 0,099]    |
| Número promedio de clientes en el nodo 3                    | 2,556   | 0,343      | [2,395; 2,716]    |
| Tiempo desde $T$ hasta que sale el último cliente ( $T_p$ ) | 44,869  | 21,948     | [34,596; 55,141]  |
| Número máximo de clientes en el sistema                     | 19,350  | 6,055      | [16,516; 22,184]  |
| Total de clientes que pasaron por el nodo 1                 | 101,750 | 11,424     | [96,403; 107,097] |
| Total de clientes que pasaron por el nodo 2                 | 40,850  | 6,335      | [37,885; 43,815]  |
| Total de clientes que pasaron por el nodo 3                 | 101,750 | 11,424     | [96,403; 107,097] |
| Total de clientes que pasaron por el sistema                | 101,750 | 11,424     | [96,403; 107,097] |



**Figura 13:** Media e intervalo de confianza del 95 % (20 repeticiones) de las seis primeras medidas, por escenario.

**Análisis de la media y la desviación estándar.** Con la convención de la sección 3.3 la tasa de llegadas es  $1/\lambda$ : 0,5 vehículos por unidad de tiempo en A y 0,333 en B, lo que explica los totales de  $151,7 \approx 0,5 \times 300$  y  $101,8 \approx 300/3$  clientes. Como todos los vehículos pasan por los nodos 1 y 3, sus totales coinciden entre sí y con el total del sistema; el nodo 2 recibe el 40 % ( $60,35/151,65 = 0,398$  y  $40,85/101,75 = 0,401$ ), lo que valida la bifurcación. Las intensidades de tráfico de cada nodo son, en A:  $\rho_1 = 0,5 \times 2,1 = 1,05$ ,  $\rho_2 = 0,2 \times 2,4 = 0,48$  y  $\rho_3 = 0,5 \times 4,1 = 2,05$  (o 1,5 con el servicio acelerado); en B:  $\rho_1 = 0,333 \times 3 = 1,0$ ,  $\rho_2 = 0,133 \times 1,5 = 0,2$  y  $\rho_3 = 0,333 \times 4 = 1,33$  (o 0,67 con el servicio acelerado).

Estas cifras explican los resultados. En A el nodo 3 es un cuello de botella severo: aun con el servicio acelerado sigue saturado ( $\rho_3 = 1,5$ ), de modo que  $n_3(t)$  crece casi linealmente hasta superar los 40 vehículos (Figura 11), el número medio en el nodo es 17,5, el tiempo medio en el sistema alcanza 99,5 unidades (un tercio del horizonte) y la red tarda  $T_p \approx 182$  unidades en vaciarse, el 61 % de  $T$ . El nodo 1, en el límite de estabilidad ( $\rho_1 = 1,05$ ), oscila entre colas cortas y largas (media 4,4 pero con desviación 2,9) y el nodo 2 está muy holgado (0,17 vehículos en promedio). En B el nodo 3 se *autorregula*: cuando  $n_3 \geq 5$  el servicio pasa a durar 2 en vez de 4 y la intensidad cae a 0,67, por lo que la cola vuelve a bajar;  $n_3(t)$  oscila alrededor de 5 (Figura 12), su media es 2,56 con la menor variabilidad relativa de todas las medidas continuas (13 %) y el nodo crítico pasa a ser la ventanilla ( $\rho_1 = 1,0$ ), que concentra la mayor parte de la espera. El resultado es un tiempo medio en el sistema tres veces menor (33,2 frente a 99,5) y un  $T_p$  cuatro veces menor (44,9 frente a 181,9), a pesar de que B tiene tiempos de servicio mayores en el nodo 1.

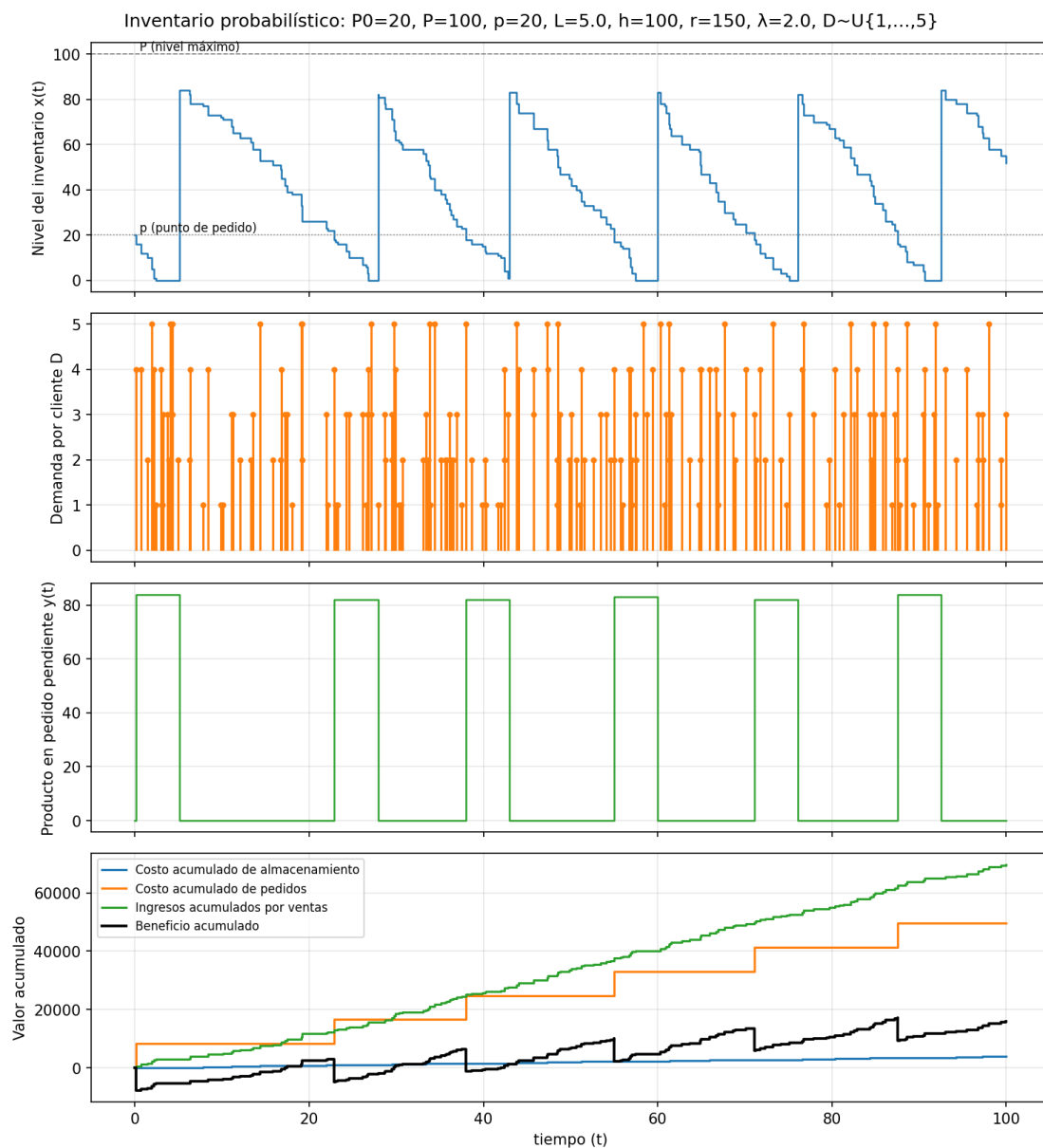


Las desviaciones estándar son grandes en las medidas que dependen de un nodo cercano a  $\rho = 1$  (el coeficiente de variación de **n\_med\_n1** es del 65 % en A y del 64 % en B, y el de  $T_p$  del 22 % y 49 %), porque un sistema en el borde de la estabilidad amplifica la aleatoriedad: unas pocas llegadas seguidas o servicios largos forman una cola que tarda mucho en disolverse. En cambio los contadores (totales por nodo) tienen coeficientes de variación del 9 % al 15 %, propios de una variable de Poisson con media  $\approx 150$  o  $\approx 100$  ( $\sqrt{150}/150 \approx 8$  %). Una sola corrida, como la de la Tabla 4, puede alejarse bastante de la media: la de A dio  $T_p = 206,7$  frente a la media de 181,9 y la de B un  $n_1$  medio de 6,4 frente a 3,6.

**Análisis de los intervalos de confianza.** Con 20 réplicas el semiancho del IC del 95 % es  $2,093 S/\sqrt{20} = 0,468 S$ . Para las medidas de flujo (totales) y para **n\_med\_n2** y **n\_med\_n3** los intervalos son estrechos (semianchos del 4 % al 8 % de la media), lo que indica que 20 réplicas bastan para estimarlas con buena precisión. Para **n\_med\_n1** y  $T_p$  los intervalos son anchos (semiancho del 30 % en **n\_med\_n1** de ambos escenarios y del 23 % en  $T_p$  de B): para reducirlos a la mitad habría que cuadruplicar las réplicas, pues el semiancho decrece como  $1/\sqrt{n}$ . Como los intervalos de A y B no se solapan en ninguna de las seis medidas continuas de la Figura 13 (en particular en el tiempo medio en el sistema, [88,6; 110,5] frente a [28,0; 38,3], y en  $T_p$ , [163,5; 200,4] frente a [34,6; 55,1]), puede afirmarse con un 95 % de confianza que el escenario B tiene menores tiempos de permanencia, menores colas en el nodo 3 y menor tiempo de vaciado que el escenario A. La única medida en la que los intervalos se solapan es **n\_med\_n1** ([3,05; 5,73] frente a [2,52; 4,65]), de modo que no hay evidencia de que la cola en ventanilla difiera entre escenarios, algo coherente con que  $\rho_1$  sea  $\approx 1$  en ambos. Debe recordarse que estos intervalos suponen réplicas independientes (garantizado por las semillas distintas) y normalidad aproximada de las medias (razonable para  $n = 20$  por el teorema central del límite), y que en el escenario A describen un sistema transitorio que depende de  $T$ .

#### 4.3. Inventario probabilístico (5.3)

La Figura 14 muestra la evolución del inventario en la corrida con los parámetros de la guía y los supuestos de la sección 3.4, y la Tabla 7 las medidas de desempeño de esa corrida y de 20 repeticiones.



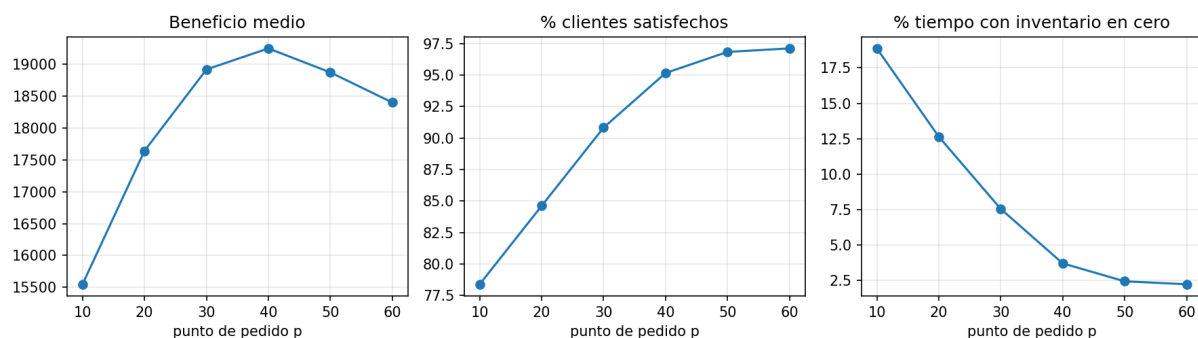
**Figura 14:** Inventario probabilístico ( $P_0 = 20$ ,  $P = 100$ ,  $p = 20$ ,  $L = 5$ ,  $h = 100$ ,  $r = 150$ ;  $\lambda = 2$ ,  $D \sim U\{1, \dots, 5\}$ ,  $c_a = 1$ ,  $T = 100$ ): nivel de inventario, demanda por cliente, cantidad en pedido pendiente y costos, ingresos y beneficio acumulados.



**Tabla 7:** Medidas de desempeño del inventario: corrida única y 20 repeticiones.

| Medida                                      | Corrida única | 20 repeticiones<br>(media $\pm$ d.e.) |
|---------------------------------------------|---------------|---------------------------------------|
| Cientes atendidos                           | 188           | —                                     |
| Ingresos por ventas                         | 69 750,00     | —                                     |
| Costo total por pedidos                     | 49 700,00     | —                                     |
| Costo total por almacenamiento              | 4 021,37      | —                                     |
| Beneficio                                   | 16 028,63     | 17 642,8 $\pm$ 3 151,8                |
| Porcentaje de clientes satisfechos          | 88,30 %       | 84,6 $\pm$ 4,3 %                      |
| Porcentaje de tiempo con inventario en cero | 9,26 %        | 12,6 $\pm$ 3,7 %                      |

La trayectoria reproduce la Figura 1 de la guía: el inventario descende a saltos con cada venta, al cruzar  $p = 20$  se dispara un pedido de  $P - x \approx 80$  unidades (tercer panel), durante el plazo  $L = 5$  el nivel sigue bajando y suele llegar a cero, y al llegar el pedido el nivel salta hasta  $P$ . Como  $P_0 = p$ , el primer pedido se lanza con la primera venta. El ciclo de reposición dura unas 16 unidades de tiempo ( $\approx 80$  unidades vendidas a 6 por unidad de tiempo), por lo que en  $T = 100$  hay seis pedidos. El beneficio acumulado (panel inferior) tiene forma de diente de sierra: cae  $h(P - x) \approx 8000$  en cada pedido y se recupera con las ventas, con margen bruto  $r - h = 50$  por unidad; el costo de almacenamiento es pequeño ( $\approx 4000$ , un 6 % de los ingresos) frente al de compras.



**Figura 15:** Sensibilidad al punto de pedido  $p$  (medias de 20 repeticiones): beneficio, porcentaje de clientes satisfechos y porcentaje de tiempo con inventario en cero.

El análisis de sensibilidad de la Figura 15 muestra el compromiso clásico del modelo: con  $p = 20$  se pierde el 15 % de los clientes y el almacén está vacío el 13 % del tiempo porque la demanda esperada durante el plazo de entrega ( $\lambda E[D] L = 30$ ) supera el punto de pedido; al subir  $p$  el porcentaje de clientes satisfechos crece hasta el 97 % y el tiempo sin existencias cae por debajo del 3 %, pero el inventario medio y su costo de almacenamiento aumentan. El beneficio medio es máximo cerca de  $p = 40$  (19 252 frente a 17 643 con  $p = 20$ , un 9 % más) y vuelve a bajar para  $p \geq 50$ . Un criterio razonable para fijar  $p$  es la demanda esperada durante  $L$  más un margen de seguridad, que aquí sitúa el punto de pedido entre 30 y 40 unidades.

#### 4.4. Simulación de eventos discretos en SimPy (5.4)

El tutorial reprodujo las salidas de la documentación: el carro alterna cargas de 5 y viajes de 2 unidades de tiempo; con la interrupción del conductor en  $t = 3$  abandona la carga y empieza a conducir; y con la estación de dos puntos los carros 2 y 3 esperan hasta  $t = 5$  y  $t = 7$  a que se



liberen los puntos. La Tabla 8 resume los resultados de los siete ejemplos, con la semilla 42 del GLC; las salidas completas están en el notebook.

**Tabla 8:** Resumen de los siete ejemplos de SimPy implementados.

| Ejemplo               | Elementos de SimPy                                                                             | Resultado observado                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Bank Renege           | <code>Resource</code> , <code>req   timeout</code> (espera con abandono)                       | De 5 clientes, 2 fueron atendidos y 3 abandonaron la cola tras agotar su paciencia (1,8; 1,5 y 3,0 min).                                 |
| Carwash               | <code>Resource(capacity=2)</code> , generador de llegadas con <code>itertools.count</code>     | En 20 min se lavaron 6 carros; los 4 iniciales ocupan las dos máquinas en dos tandas de 5 min.                                           |
| Machine Shop          | <code>PreemptiveResource</code> , <code>Interrupt</code> , prioridades                         | 10 máquinas con fallas <code>Exp(300 min)</code> y un técnico produjeron 33 316 piezas en 4 semanas ( $\approx 3\,332$ por máquina).     |
| Movie Renege          | <code>Container</code> , eventos personalizados ( <code>env.event</code> ), abandono colectivo | Las tres películas se agotaron (a los 39,5, 36,0 y 29,0 min); 8, 6 y 13 personas abandonaron la cola al agotarse.                        |
| Gas Station Refueling | <code>Resource</code> , <code>Container</code> , proceso de control periódico                  | El camión cisterna fue llamado en $t = 980$ y $t = 1800$ s; el carro 5 esperó 324,5 s por combustible.                                   |
| Process Communication | <code>Store</code> , <code>env.all_of</code> , canal <i>broadcast</i>                          | En el canal broadcast ambos consumidores reciben cada mensaje; en el <code>Store</code> compartido los mensajes se alternan entre A y B. |
| Event Latency         | <code>Store</code> más proceso auxiliar con <code>timeout</code> (retardo)                     | Cada mensaje enviado en $t$ se recibe en $t+10$ : el emitido en $t = 5$ llega en $t = 15$ , etc.                                         |

En comparación con la implementación manual de las secciones anteriores, SimPy elimina la gestión explícita de la lista de sucesos y de las rutinas: el modelo se describe desde el punto de vista de cada entidad (“el cliente llega, pide el recurso, espera, es atendido, se va”) y el entorno se encarga de ordenar los eventos. El precio es que las variables acumuladoras (número medio en cola, tiempo medio en el sistema) deben añadirse explícitamente con *monitores*, mientras que en el enfoque de Rios08 forman parte de las rutinas. La inyección del GLC mediante una subclase de `random.Random` permitió, además, usar el mismo generador validado en ambos enfoques y reproducir exactamente cada corrida.

#### 4.5. Simulación de eventos discretos en AnyLogic (5.5)

Los cinco modelos de los tutoriales se construyeron y ejecutaron correctamente en AnyLogic PLE (Figuras 1 a 7). En *Bank Office*, en el instante de la captura habían llegado 83 clientes, de los cuales 43 fueron al cajero automático y 40 a ventanilla, con una longitud media de cola de 0,04 clientes, una utilización del cajero automático del 26 % y de los cuatro cajeros del 19 %, y un tiempo medio en el sistema de 4,17 minutos con una distribución asimétrica concentrada entre 2 y 3 minutos: el sistema está holgado, al contrario que los escenarios saturados de las secciones 5.1 y 5.2. Este modelo es, en esencia, la red de colas de la sección 5.2 construida con bloques: `source` equivale a la rutina `Llegada`, `selectOutput` a la bifurcación con probabilidad fija, `queue+ATM` y `service` a los nodos con sus rutinas de fin de servicio, y `timeMeasureStart/End` a los vectores  $LL(i)$  y  $S(i)$  con los que se calcula el tiempo medio en el sistema.

En *Subway entrance* se generaron 4731 pasajeros, el 10 % (477) pasó por las máquinas expendedoras y el mapa de densidad muestra las zonas críticas: la entrada, donde los flujos se cruzan, y la salida hacia los trenes después de los torniquetes, con densidades cercanas a 1,5



peatones/m<sup>2</sup>. En *Intersection*, 49 vehículos fueron generados por las tres fuentes y 33 habían salido en el momento de la captura; el descriptor de la red vial muestra la vía este en rojo, pues allí se acumulan los carros detenidos por el semáforo y el bus en su parada. En *Job Shop* llegaron 120 paletas, 37 ya habían sido procesadas, y la utilización de las dos máquinas CNC (72 %) es mayor que la de los cinco montacargas (46 %), lo que identifica al mecanizado como el cuello de botella. En *Lead Acid Battery Production* se produjeron 375 ánodos y 375 cátodos, se ensamblaron 165 grupos de placas y 6 baterías habían llegado a la zona de carga; los montacargas trabajan al 93 % mientras el AGV solo al 8 %, lo que sugiere que convendría reasignar transporte al AGV.

Los cuatro últimos tutoriales muestran lo que las librerías especializadas añaden al paradigma de eventos discretos: el movimiento en el espacio (navegación de peatones con evitación de colisiones, reglas de tránsito y semáforos, transportadores y vehículos con rutas), que sería muy costoso de implementar a mano con la metodología de Rios08, mientras que la lógica de proceso subyacente (fuentes, colas, servicios, bifurcaciones, sumideros) sigue siendo la misma de las secciones anteriores.

## 5. Conclusiones

- La metodología de simulación de sucesos discretos de Rios08 (estado, reloj, lista de sucesos, contadores, acumuladores y una rutina por suceso) se aplicó sin cambios estructurales a tres sistemas distintos: la línea de espera con dos sucesos, la red de colas con cuatro y el inventario con dos. Lo único que cambia entre modelos son las rutinas; el bucle principal que extrae el suceso más próximo es idéntico, lo que confirma que el paradigma es general.
- Los dos escenarios del  $G/G/1$  están saturados ( $\rho = 1,67$  en A y  $\rho \approx 2$  en B), por lo que la cola crece a tasa  $\lambda - \mu$  durante todo el horizonte y los tiempos medios dependen de  $T$ . El escenario A atiende cinco veces más clientes con colas más largas (máximo medio 78 frente a 23), pero el B hace esperar más a cada cliente (23,8 frente a 12,8 unidades) y tarda más en vaciarse ( $T_p = 44$  frente a 24) porque su servidor es seis veces más lento.
- En la red de colas el nodo 3 es el cuello de botella del escenario A incluso con el servicio acelerado ( $\rho_3 = 1,5$ ), lo que produce un tiempo medio en el sistema de 99,5 y un vaciado de 182 unidades; en el escenario B la regla de servicio dependiente de la congestión estabiliza el nodo ( $\rho_3$  pasa de 1,33 a 0,67 cuando  $n_3 \geq 5$ ) y el tiempo medio baja a 33,2. Los intervalos de confianza del 95 % con 20 réplicas no se solapan entre escenarios en ninguna medida continua salvo `n_med_n1`, de modo que la diferencia es estadísticamente significativa; los intervalos más anchos (semianchos del 23 % al 30 %) corresponden a las medidas que dependen de un nodo con  $\rho \approx 1$ , donde la varianza entre réplicas es intrínsecamente alta.
- Las desviaciones estándar muestrales mostraron que una sola corrida es una estimación pobre: en la red de colas la corrida única del escenario A dio  $T_p = 206,7$  frente a una media de 181,9, y en el inventario el beneficio de la corrida única (16 029) quedó medio desvío por debajo de la media de 20 réplicas (17 643). Replicar con semillas independientes y reportar media, desviación e IC es imprescindible.
- El inventario con política  $(p, P)$  y los parámetros de la guía ( $p = 20$ ) deja insatisfecho al 15 % de los clientes y mantiene el almacén vacío el 13 % del tiempo, porque la demanda esperada durante el plazo de entrega (30 unidades con los supuestos adoptados) supera el punto de pedido. El análisis de sensibilidad indica que  $p \approx 40$  maximiza el beneficio medio (+9 %) y eleva la satisfacción al 95 %; por encima de 50 el costo de almacenamiento vuelve a reducir el beneficio.



- SimPy y AnyLogic implementan el mismo paradigma con dos abstracciones distintas: SimPy describe el sistema como procesos que ceden eventos y delega la lista de sucesos al entorno, y AnyLogic como un diagrama de bloques que encapsulan las rutinas de suceso y añaden animación y estadísticas. Los siete ejemplos de SimPy y los cinco tutoriales de AnyLogic se construyeron y ejecutaron correctamente; en SimPy fue posible, además, reutilizar el generador congruencial propio como fuente de aleatoriedad, y en AnyLogic las estadísticas integradas (utilización, longitud de cola, histograma del tiempo en el sistema, mapas de densidad) permitieron identificar de inmediato los cuellos de botella de cada modelo.

## Bibliografía

- [1] A. A. Cruz Roa, “6. simulación de eventos discretos,” Diapositivas del curso Simulación Computacional, Universidad de los Llanos, 2023.
- [2] D. Ríos Insua, S. Ríos Insua, and J. Martín Jiménez, *Simulación. Métodos y aplicaciones*, 2nd ed. México: Alfaomega / Ra-Ma, 2008.
- [3] Team SimPy, “SimPy: Discrete event simulation for Python — documentation (release 4.1),” <https://simpy.readthedocs.io/en/latest/>, 2024, consultado en septiembre de 2026.
- [4] The AnyLogic Company, “AnyLogic tutorials: Bank office, subway entrance, road traffic, job shop, material handling,” <https://anylogic.help/tutorials/>, 2024, consultado en septiembre de 2026.
- [5] T. E. Hull and A. R. Dobell, “Random number generators,” *SIAM Review*, vol. 4, no. 3, pp. 230–254, 1962.
- [6] G. Marsaglia and T. A. Bray, “A convenient method for generating normal variables,” *SIAM Review*, vol. 6, no. 3, pp. 260–264, 1964.